ESCOLA SUPERIOR DE TECNOLOGIA UPCA

INTEGRAÇÃO DE SISTEMAS DE INFORMAÇÃO

TRABALHO PRÁTICO I

Licenciatura em Engenharia de Sistemas Informáticos

Alunos

Edgar Casal (a31026)

João Simão Machado (a31510)

Professor

Óscar Ribeiro

outubro de 2026

1. Enquadramento **2**
2. Problema **4**
2.1 Contexto 4
2.2 Dados disponíveis 4
2.3 Desafios 4
2.4 Resultado pretendido 5
3. Estratégia utilizada **6**
3.1 Arquitetura da solução 6
3.2 Fontes de dados e formatos 6
3.3 Ferramentas e divisão do trabalho 7
3.4 Modelo de dados 7
3.5 Operadores utilizados por fase 8
3.6 Correspondência com os requisitos do enunciado 8
4. Transformações **10**
4.1 KNIME 10
4.1.1 Leitura e correção do CSV 10
4.1.2 Normalização e seleção de candidatos 11
4.1.3 API Open Library 11
4.1.4 Edição em português e duplicados 12
4.1.5 Classificação por género 13
4.1.6 Ranking 13
4.1.7 Preparação para a base de dados 14
4.2 Pentaho Data Integration (Kettle) 15
4.2.1 Limpeza, normalização e seleção de candidatos (t1_limpeza) 15
4.2.2 Consulta à Open Library, duplicados e carga final (t2_api) 16
5. Jobs **17**
5.1 KNIME 17
5.1.1 Carga na base de dados com tratamento de erros 17
5.1.2 Log, exportação e notificação 18
5.2 Dashboard em Node-RED 19
5.3 Pentaho Data Integration (Kettle) 20
6. Vídeo com demonstração **22**
7. Conclusão e trabalhos futuros **24**
7.1 Conclusão 24
7.2 Limitações 25
7.3 Trabalhos futuros 25
8. Referências **26**

1. Enquadramento

O presente relatório documenta o Trabalho Prático I da unidade curricular de Integração de Sistemas de Informação (ISI), do 3.º ano da Licenciatura em Engenharia de Sistemas Informáticos da Escola Superior de Tecnologia da Universidade Politécnica do Cávado e do Ave (UPCA), no ano letivo 2026/27, sob orientação do docente Óscar Ribeiro.

O trabalho foi realizado por um grupo de dois elementos:

Nome	Número	Plataforma explorada
Edgar Casal	a31026	KNIME Analytics Platform
João Simão Machado	a31510	Pentaho Data Integration (Kettle)

O objetivo do trabalho é aplicar e experimentar ferramentas de suporte a processos de ETL (*Extract, Transform and Load*), inerentes à integração de sistemas de informação ao nível dos dados. Pretende-se desenvolver um processo completo que recolha dados de fontes e formatos heterogéneos, os limpe, valide, transforme e cruze, e os disponibilize em diferentes destinos: base de dados, ficheiros de exportação, notificações por email e um dashboard.

O tema escolhido pelo grupo foi livros e avaliações. A partir do dataset público *Goodreads-books*, disponível no Kaggle, e da API pública da Open Library, o processo desenvolvido apoia uma editora portuguesa (cenário imaginário) a identificar livros populares e bem avaliados que ainda não têm edição em português.

Para além das ferramentas de ETL, foram utilizadas as seguintes tecnologias complementares:

- **PostgreSQL**, como base de dados de destino e ponto de integração entre as várias partes do sistema;
- **Node-RED**, com o módulo Dashboard 2.0, para a visualização dos resultados;
- **Git / GitHub**, para o controlo de versões e a partilha do trabalho entre os elementos do grupo.

Tal como pedido no enunciado, cada elemento do grupo explorou uma plataforma diferente. O processo completo foi desenvolvido em KNIME. Em Kettle foi desenvolvida uma versão reduzida do mesmo processo, o que permite comparar as duas ferramentas sobre os mesmos dados.

O documento está organizado da seguinte forma:

- a **secção 2** descreve o problema a resolver;

- a **secção 3** apresenta a estratégia adotada e a arquitetura da solução;
- as **secções 4 e 5** detalham, respetivamente, as transformações e os jobs desenvolvidos em cada plataforma;
- a **secção 6** contém o acesso ao vídeo de demonstração;
- a **secção 7** apresenta as conclusões, as limitações e os trabalhos futuros;
- a **secção 8** lista as referências utilizadas.

2. Problema

2.1 Contexto

Uma editora portuguesa (cenário imaginário) pretende alargar o seu catálogo com traduções de livros estrangeiros. Escolher que livros traduzir é uma decisão com impacto financeiro relevante: os direitos de tradução, a tradução e a edição representam um investimento significativo. Por isso, a editora procura apoiar essa decisão em dados, em vez de depender apenas da intuição dos editores.

A questão central que o trabalho pretende responder é:

Que livros populares e bem avaliados pelos leitores ainda não têm edição em português, e quais devem ser considerados prioritários em cada género?

2.2 Dados disponíveis

Para responder a esta questão foram usadas duas fontes de dados públicas, de natureza e formato diferentes:

- **Dataset *Goodreads-books* (Kaggle):** um ficheiro CSV com 11 127 livros e 12 atributos, entre os quais o título, os autores, a avaliação média, o número de avaliações, os ISBN, o idioma, o número de páginas, a data de publicação e a editora. É a fonte principal sobre a popularidade e a avaliação dos livros.
- **API de pesquisa da Open Library:** um serviço web REST que, a partir do ISBN de um livro, devolve em formato JSON informação sobre a obra, nomeadamente os idiomas de todas as edições conhecidas, o número de edições e os temas (*subjects*). É esta fonte que permite saber se um livro já tem edição em português, informação que não existe no dataset do Goodreads.

2.3 Desafios

A integração destas fontes levanta vários problemas de qualidade e de compatibilidade dos dados:

- **Linhas mal formatadas:** algumas linhas do CSV têm vírgulas dentro do nome dos autores, o que impede a leitura direta do ficheiro.
- **Inconsistência nos códigos de idioma:** o mesmo idioma aparece com códigos diferentes (por exemplo, o inglês surge como `eng`, `en-US`, `en-GB` e `en-CA`), e os códigos usados pela Open Library diferem dos do Goodreads.
- **Edições duplicadas:** a mesma obra aparece várias vezes, em edições diferentes, com ISBN distintos.
- **Tipos e formatos:** os ISBN têm de ser tratados como texto (podem terminar em "X"), e as datas estão no formato americano (mês/dia/ano), em texto.
- **Ausência de géneros:** o dataset não tem informação sobre o género dos livros. Os temas devolvidos pela API são numerosos, pouco uniformes e estão em várias línguas.

- **Cobertura da fonte externa:** a Open Library é mantida de forma colaborativa e pode não ter registo de todas as edições existentes, o que afeta a fiabilidade da verificação.

2.4 Resultado pretendido

O processo de ETL desenvolvido deve:

1. recolher e corrigir os dados do Goodreads, garantindo que nenhum livro se perde
2. selecionar os livros populares e bem avaliados
3. enriquecê-los com a informação da Open Library, para identificar os que não têm edição em português registada
4. classificar esses livros por género e ordená-los por uma pontuação que tenha em conta a avaliação e o número de avaliações
5. disponibilizar os resultados numa base de dados PostgreSQL, em ficheiros de exportação (CSV e JSON), num email de resumo e num dashboard
6. registar cada execução e reagir a erros, de forma a poder ser executado repetidamente de forma fiável.

O resultado final é uma **pré-seleção de candidatos a tradução por género**, que serve de ponto de partida à análise dos editores.

3. Estratégia utilizada

3.1 Arquitetura da solução

A solução segue a estrutura clássica de um processo ETL. Os dados são recolhidos de fontes heterogéneas, transformados e enriquecidos em KNIME, e carregados numa base de dados PostgreSQL. A base de dados é o ponto de integração entre as várias partes do sistema: é a partir dela que são gerados os ficheiros de exportação, o email de resumo e o dashboard em Node-RED.

O processo foi organizado em três fases:

- **Extração:** leitura do dataset do Goodreads, consulta da API da Open Library e leitura das regras de classificação de géneros.
- **Transformação:** correção, validação e normalização dos dados, seleção dos candidatos, enriquecimento com a informação da API, remoção de duplicados, classificação por género e cálculo do ranking.
- **Carregamento:** inserção dos resultados na base de dados, exportação de ficheiros, registo das execuções, notificação por email e visualização num dashboard.

3.2 Fontes de dados e formatos

Fonte	Tipo	Formato	Utilização
`books.csv` (Kaggle)	Ficheiro local	CSV	Dados principais dos livros (avaliações, ISBN, idioma, editora…)
Open Library Search API	Serviço web REST	JSON	Idiomas das edições, n.º de edições, temas e identificador da obra
`generos.xml`	Ficheiro de configuração criado pelo grupo	XML	Regras (expressões regulares) de classificação por género
PostgreSQL (`goodreads_etl`)	Base de dados relacional	SQL	Destino dos dados e fonte para as exportações e o dashboard

Os resultados são disponibilizados em vários formatos: tabelas na base de dados, ficheiros CSV e JSON com o ranking, um ficheiro CSV com o histórico de execuções, um email com o ranking em anexo e um dashboard web.

3.3 Ferramentas e divisão do trabalho

Ferramenta	Utilização
KNIME Analytics Platform	Processo ETL completo (elemento: Edgar Casal)
Pentaho Data Integration (Kettle)	Versão reduzida do processo, para comparação (elemento: João Simão Machado)
PostgreSQL 18	Base de dados de destino
Node-RED + Dashboard 2.0	Dashboard de visualização dos resultados
Git / GitHub	Controlo de versões e partilha do trabalho

Para que as duas plataformas não interfiram uma com a outra, os dados carregados pelo KNIME ficam no schema `public` da base de dados, e os carregados pelo Kettle ficam num schema próprio (`kettle`). Isto permite comparar diretamente os resultados das duas ferramentas.

3.4 Modelo de dados

A base de dados `goodreads_etl` tem a seguinte estrutura:

- **`livros`:** os 11 127 livros do dataset, já corrigidos e normalizados.
- **`candidatos`:** os candidatos a tradução, com o género, a pontuação e a posição no ranking. Está ligada a `livros` pelo `book_id` (chave estrangeira), para não repetir informação.
- **`log_execucoes`:** uma linha por cada execução do processo, com as contagens, o estado (sucesso ou erro) e uma mensagem.
- **`v_ranking`:** uma vista que junta `candidatos` e `livros` e apresenta o ranking final, ordenado por género e posição.

A estrutura é criada pelo script `schema.sql`, incluído no repositório.

3.5 Operadores utilizados por fase

Fase	Principais operadores (nós KNIME)
Extração	Line Reader, CSV Reader, GET Request, JSON Path, XML Reader, XPath, DB Query Reader
Transformação	String Manipulation (regex), Row Filter, Duplicate Row Filter, GroupBy, Ungroup, Joiner, Cross Joiner, Rule Engine, String to Date&Time, Math Formula, Rank, Sorter, Column Filter, Column Renamer
Carregamento	DB SQL Executor, DB Row Inserter, CSV Writer, Table to JSON, JSON Writer, Send Email
Orquestração	Flow variables, Try / Catch Errors, Rule Engine Variable, Merge Variables, Credentials Configuration

3.6 Correspondência com os requisitos do enunciado

Requisito	Como foi cumprido
Diversidade de formatos	CSV, JSON e XML na entrada; CSV, JSON e base de dados na saída
Diversidade de fontes	Ficheiro local, API REST e base de dados relacional
Expressões regulares	Correção de linhas mal formatadas, normalização de idiomas, classificação por género e validação de datas
Joins, agregações e lookups	Left outer join com os dados da API, cross join com as regras de géneros, GroupBy para contagens e validações
Operações sobre valores	Normalizar, filtrar, converter, calcular, ordenar, remover duplicados

Acesso a APIs remotas	Open Library Search API
Base de dados	PostgreSQL, com tabelas, chave estrangeira, vista e carga completa
Jobs / orquestração	Ordem de execução por flow variables, tratamento de erros com Try/Catch, email de sucesso e de alerta
Logs	Tabela `log_execucoes` e exportação para ficheiro
Visualização	Dashboard em Node-RED a ler da base de dados

A anonimização de dados não foi aplicada, porque o dataset não contém dados pessoais: os autores são figuras públicas, e não existe informação sobre leitores.

4. Transformações

4.1 KNIME

O processo foi organizado em metanós, um por etapa, apresentados pela ordem de execução. A visão geral do workflow é apresentada na secção 5. Em cada etapa foi incluída uma validação, com contagens que permitem confirmar que nenhum dado se perde nem é duplicado indevidamente.

4.1.1 Leitura e correção do CSV

A leitura direta do `books.csv` falhava em 4 linhas, que tinham vírgulas dentro do nome dos autores (por exemplo, “Sam Bass Warner, Jr.”). Para não alterar a fonte original, o ficheiro é lido linha a linha (Line Reader) e corrigido com uma expressão regular, `", ([^ ])"` → `" $1"`, que remove apenas as vírgulas seguidas de um espaço e de outro carácter. Uma primeira regra mais simples apanhava também 2 falsos positivos (o cabeçalho e um título começado por espaços), detetados com um Row Filter de validação. O resultado é gravado numa área de *staging* (`books_clean.csv`) e lido de novo com o CSV Reader. A ordem de execução entre a escrita e a leitura é garantida por uma ligação de flow variables. Os ISBN são tratados como texto.

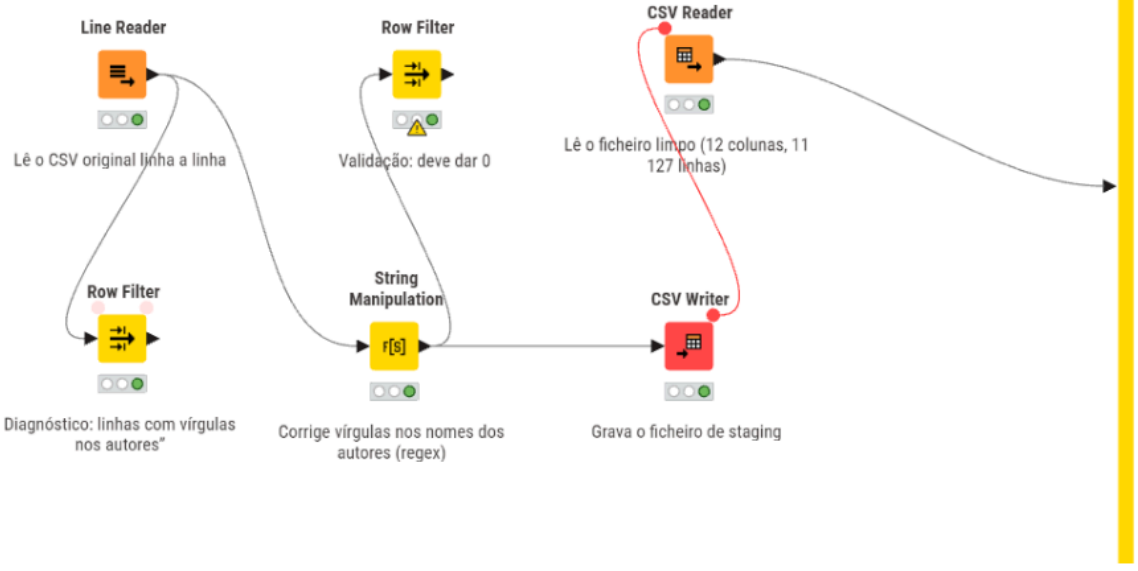


Figura 1: Metanó “Leitura e correção do CSV”

Validação: 4 linhas com problemas antes da correção, 0 depois; 11 127 livros e 12 colunas lidos.

4.1.2 Normalização e seleção de candidatos

O idioma aparecia com 27 códigos diferentes, e o inglês estava dividido em quatro (`eng`, `en-US`, `en-GB` e `en-CA`). A expressão regular `^en(g|-[A-Z]+)$` converte as variantes em `eng`, mantendo de fora o inglês médio (`enm`). De seguida são selecionados os candidatos: livros em inglês, com pelo menos 50 000 avaliações e avaliação média igual ou superior a 4.

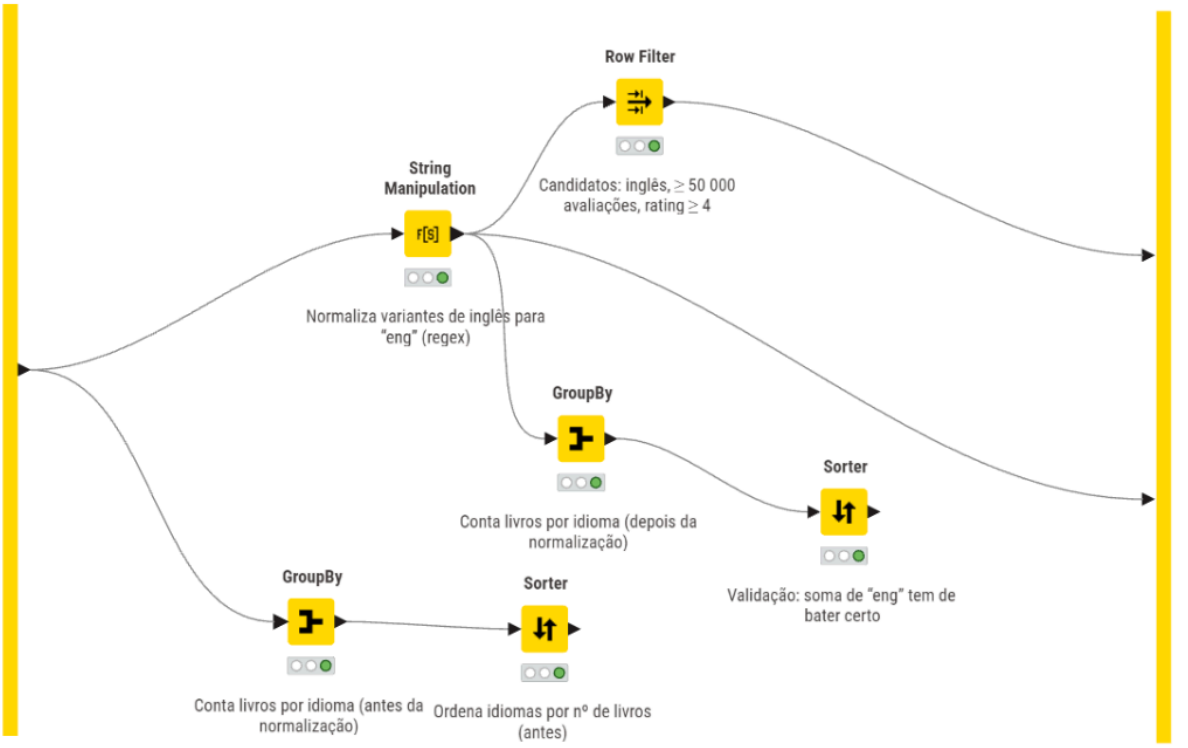


Figura 2: Metanó "Normalização e candidatos"

Validação: 27 → 24 códigos; `eng` passa a 10 541 livros (8 911 + 1 409 + 214 + 7); o total mantém-se em 11 127; 363 candidatos selecionados.

4.1.3 API Open Library

Para cada candidato é construído um URL de pesquisa por ISBN, e a API é chamada com o nó GET Request, com um cabeçalho `User-Agent` identificado e um intervalo de 1 segundo entre pedidos. O nó JSON Path extrai da resposta o número de resultados, o número de edições, os idiomas e os temas de todas as edições, e o identificador da obra.

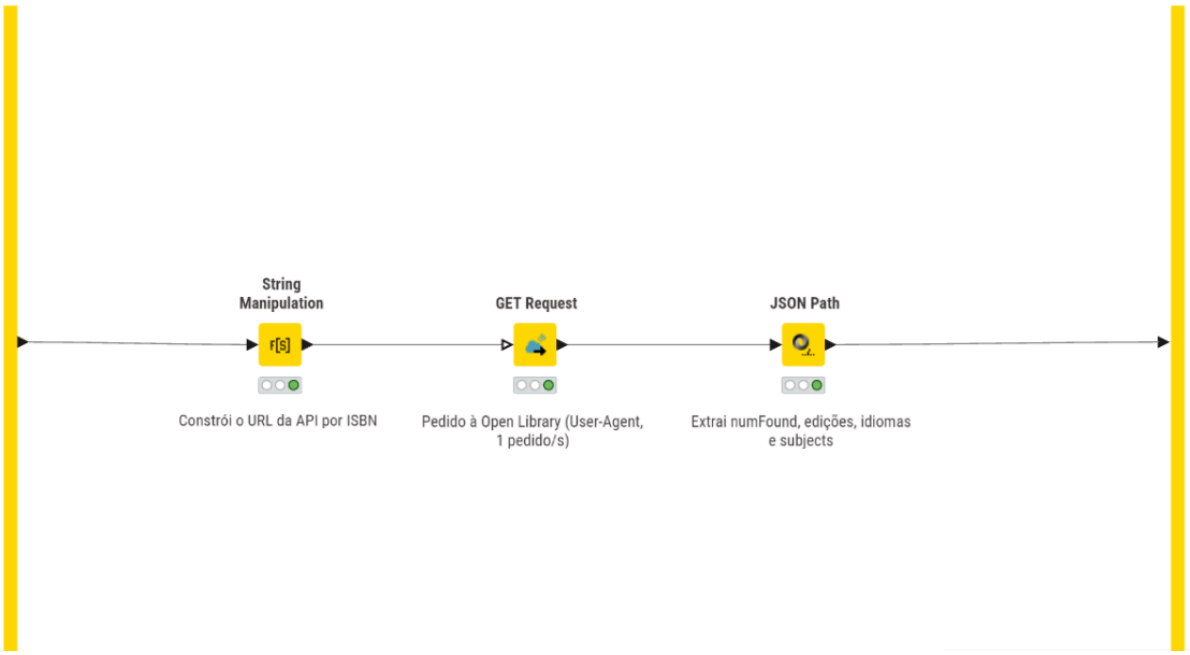


Figura 3: Metanó "API Open Library"

Validação: respostas confirmadas manualmente no browser para uma amostra de livros, incluindo casos com 0 e com 2 resultados.

4.1.4 Edição em português e duplicados

A lista de idiomas é "explodida" com o nó Ungroup (uma linha por livro e idioma), filtrada pelo código `por` e agregada por livro. O resultado é juntado à tabela principal com um *left outer join*, e um Rule Engine classifica cada livro como "Sim", "Não" ou "Desconhecido". As edições repetidas da mesma obra são removidas com o identificador da obra na Open Library, mantendo a edição com mais avaliações. Por fim, são selecionados os livros sem edição em português registada.

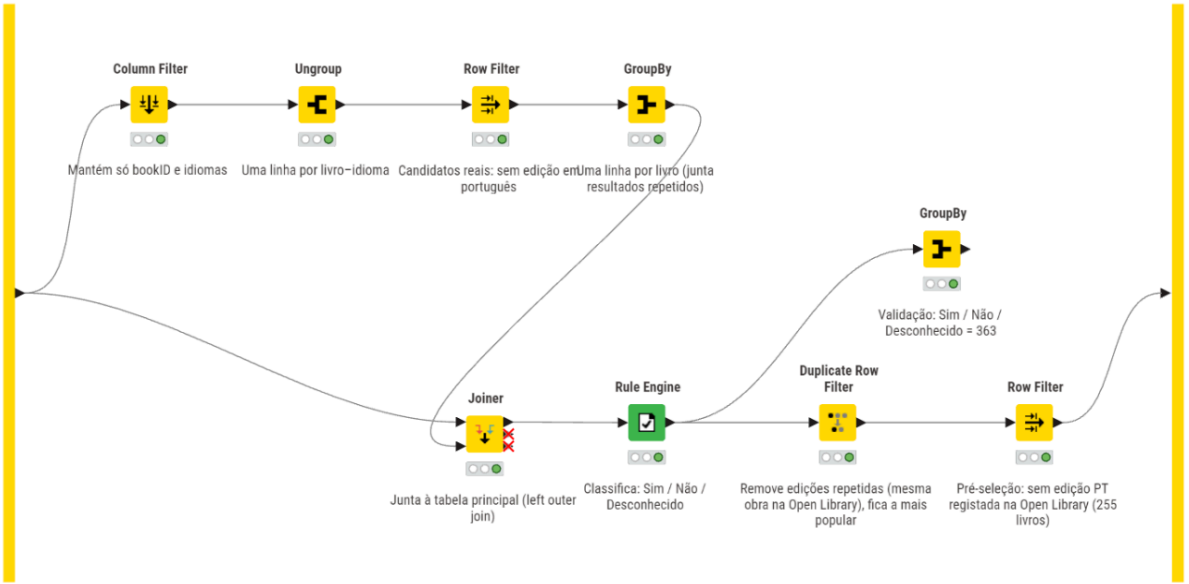


Figura 4: Metanó "Edição PT e duplicados"

Validação: 107 “Sim”, 255 “Não” e 1 “Desconhecido” (total 363); 3 duplicados removidos; 255 candidatos finais. Uma verificação manual de 7 livros classificados como “Não” revelou que 6 tinham edição em português, o que mostra a cobertura limitada da Open Library (ver secção 7).

4.1.5 Classificação por género

As regras de classificação estão num ficheiro externo, `generos.xml`, com 8 géneros e um padrão de expressão regular para cada um. O ficheiro é lido com o XML Reader e o XPath. Cada tema de cada livro é comparado com cada regra através de um Cross Joiner e da função `regexMatcher`, e cada livro fica com o género que teve mais correspondências. Os livros sem nenhuma correspondência ficam como “Outro”. Durante a validação foram encontrados e corrigidos dois falsos positivos, apenas no XML e sem alterar o workflow:

- `science` apanhava “science fiction”, e passou a `science(?! fiction)`;
- `war` apanhava “warlocks” e “award”, e passou a `\bwar\b`.

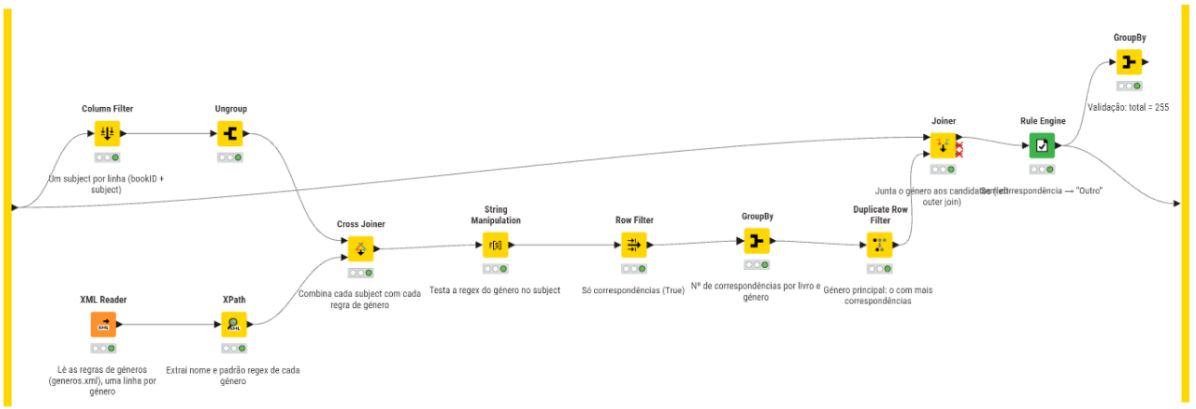


Figura 5: Metanó “Géneros”

Validação: a distribuição por género soma 255 livros; 25 ficaram em “Outro” (cerca de 10%).

4.1.6 Ranking

A pontuação de cada candidato usa a média ponderada originalmente usada pelo IMDb no seu top 250, que penaliza as avaliações médias suportadas por poucas avaliações:

```
pontuação = (v / (v + m)) × R + (m / (v + m)) × C
```

Nesta fórmula, `R` é a avaliação média do livro, `v` o número de avaliações, `m` = 50 000 (o mínimo exigido) e `C` = 3,943, a média geral, calculada no próprio workflow a partir dos livros com avaliações e passada ao Math Formula como flow variable. O nó Rank atribui a posição dentro de cada género, usando o número de edições como critério de desempate.

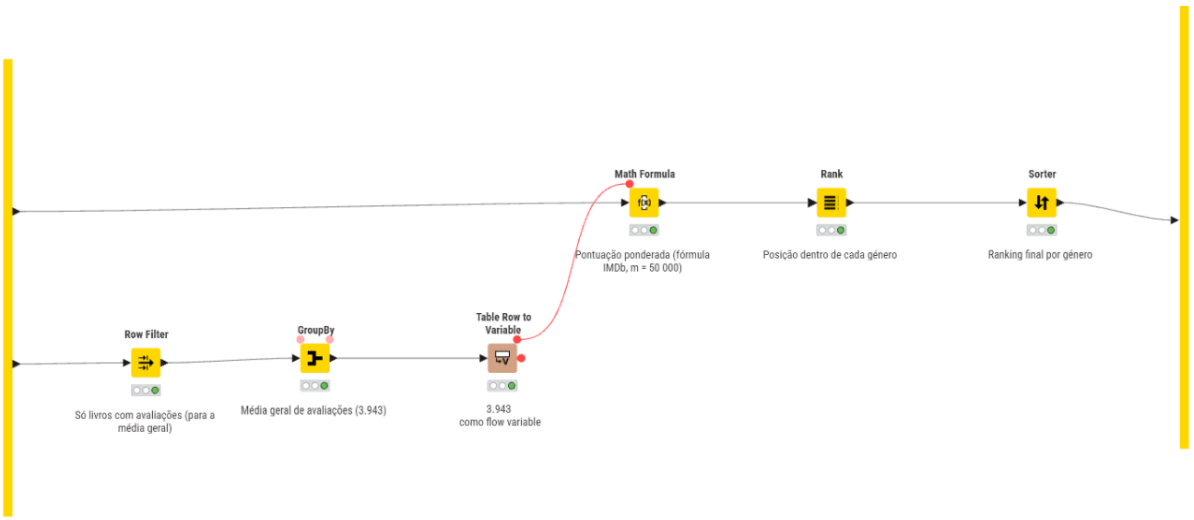


Figura 6: Metanó "Ranking"

Validação: 255 livros ordenados. Por exemplo, em Biografia, "Persepolis 2" (média 4,24, com 57 mil avaliações) fica em 8.º lugar, abaixo de "Into Thin Air" (4,17, com 351 mil avaliações), como pretendido.

4.1.7 Preparação para a base de dados

Antes da carga, os títulos são limpos de espaços no início e no fim, as datas de publicação são convertidas de texto (formato `M/d/yyyy`) para o tipo data, e as colunas são selecionadas e renomeadas para corresponderem às tabelas `livros` e `candidatos`.

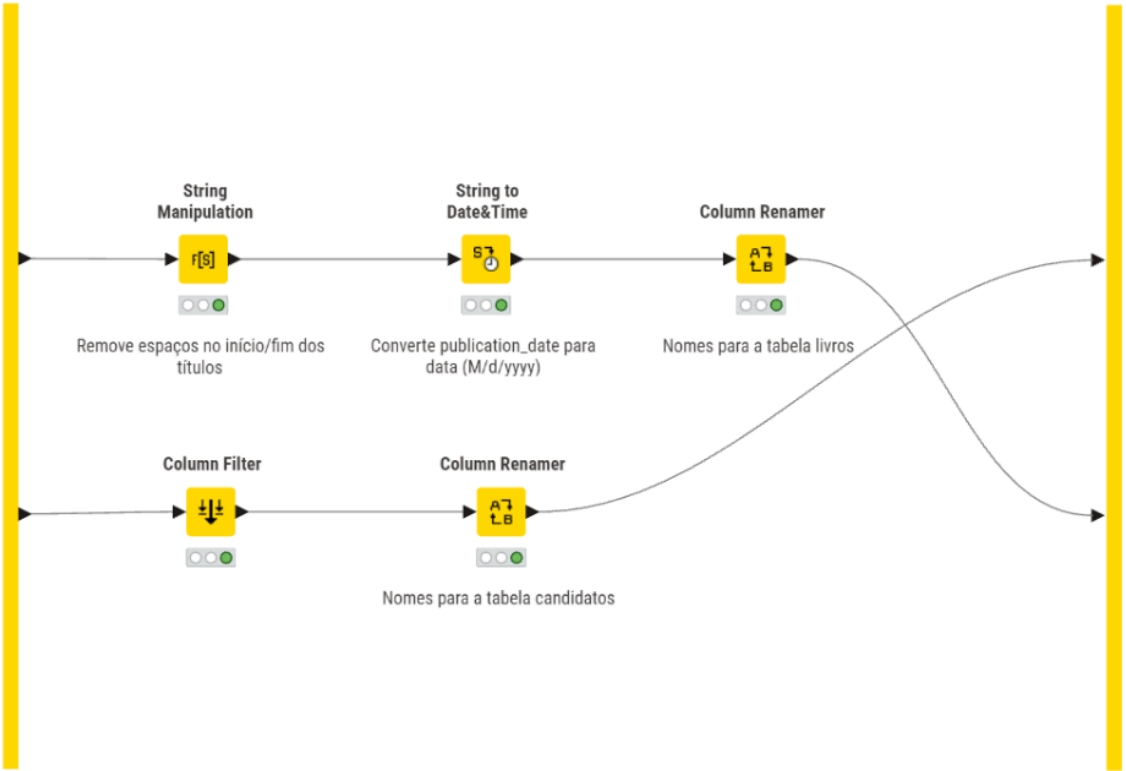


Figura 7: Metanó "Preparação para a BD"

Validação: colunas e tipos coincidentes com o `schema.sql`, confirmados pela carga sem erros (secção 5).

4.2 Pentaho Data Integration (Kettle)

A versão em Kettle é uma versão reduzida do processo. Cobre a leitura e correção do CSV, a normalização dos idiomas, a seleção dos candidatos, a consulta à Open Library, a classificação da edição em português, a remoção de duplicados e a carga na base de dados, mas não inclui a classificação por gênero nem o ranking. É composta por duas transformações (ficheiros `.ktr`), executadas em sequência pelo job descrito na secção 5.3. Ambas usam a ligação partilhada `goodreads_pg` e gravam no schema `kettle`, separado do schema `public` onde o KNIME carrega os seus dados. As credenciais da base de dados e a pasta de dados são definidas em variáveis (`PG_USER`, `PG_PASS` e `DATA_DIR`, no ficheiro `kettle.properties`), para não ficarem gravadas nos ficheiros do projeto.

4.2.1 Limpeza, normalização e seleção de candidatos (t1_limpeza)

A transformação `t1_limpeza` lê o `books.csv` linha a linha com o step `Text file input` (sem separador nem delimitador de texto, de modo a cada linha ficar num único campo, e com o ficheiro marcado como obrigatório). Um `Replace in string` aplica a expressão regular `, ([^ ])` → `$1`, que remove apenas as vírgulas dentro do nome dos autores e evita os dois falsos positivos (o cabeçalho e o livro 6549). O `Split Fields` parte depois a linha em 12 colunas com nomes dados explicitamente, o que resolve também a coluna `num_pages` com espaços no nome, e o `Select values` define os tipos: os campos numéricos passam a inteiros e decimais, e `isbn`, `isbn13` e `publication_date` ficam como texto. Um segundo `Replace in string` normaliza o idioma com a expressão `^en(g|-[A-Z]+)$` → `eng`. Um `Filter rows` seleciona os candidatos (idioma `eng`, `ratings_count` ≥ 50 000 e `average_rating` ≥ 4), que são gravados com `Table output` na tabela `kettle.candidatos`, com a opção de esvaziar a tabela antes de cada carga (TRUNCATE). Um ramo lateral com `Memory Group By` (em modo cópia, sem afetar o fluxo principal) serve só para validar a contagem por idioma.

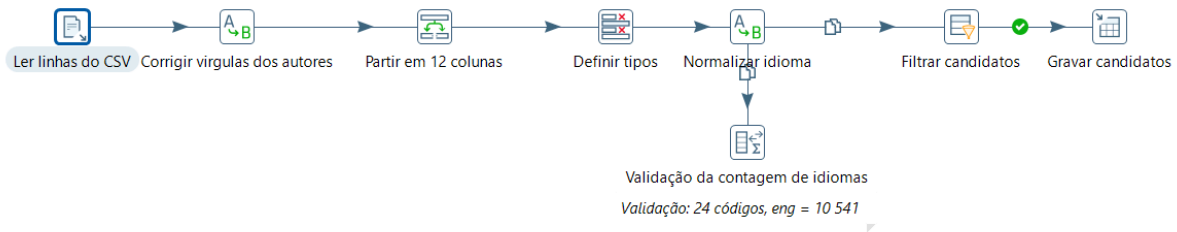


Figura 8: Transformação "t1_limpeza" em Kettle

Validação: 11 127 livros e 12 colunas depois de corrigir as linhas, sem erros; 27 → 24 códigos de idioma; `eng` passa a 10 541 livros (8 911 + 1 409 + 214 + 7); 363 candidatos

selecionados e gravados em `kettle.candidatos`. Uma segunda execução continua a dar 363 linhas (e não 726), o que confirma que o TRUNCATE funciona.

4.2.2 Consulta à Open Library, duplicados e carga final (t2_api)

A transformação `t2_api` lê os candidatos de `kettle.candidatos` com um `Table input`, que constrói o URL de pesquisa por ISBN (`isbn13`) na própria consulta SQL. Um `Add constants` define o cabeçalho User-Agent identificado e um `Delay row` impõe um intervalo de 1 segundo por pedido. O `REST client` (com uma única cópia, para os pedidos não correrem em paralelo) chama a API e guarda a resposta JSON. Um `Modified JavaScript Value` lê o JSON, junta os idiomas de todos os resultados (necessário quando `numFound` = 2), guarda o identificador da obra (`docs[0].key`) e classifica cada livro como "Sim" (a lista de idiomas inclui `por`), "Não" ou "Desconhecido" (`numFound` = 0). Depois de retiradas as colunas técnicas (`Select values`), os livros são ordenados por obra e por número de avaliações decrescente (`Sort rows`) e o `Unique rows` mantém, em cada obra, a edição com mais avaliações. O resultado é gravado em `kettle.livros_final` (com TRUNCATE) e, num ramo paralelo em modo cópia, exportado para Excel (`Microsoft Excel Writer`, ficheiro `livros_candidatos.xlsx`). Tal como na `t1_limpeza`, há um ramo lateral só para validar a contagem por classificação.

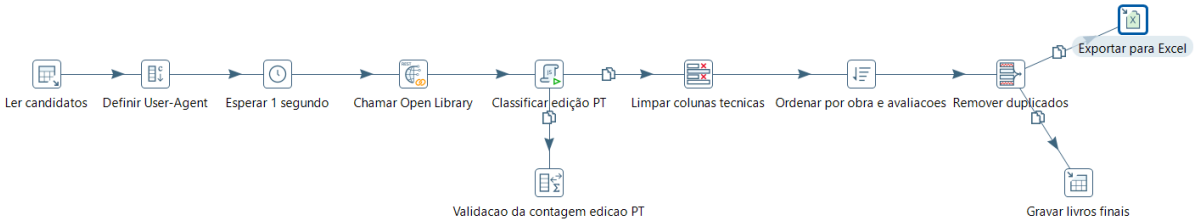


Figura 9: Transformação "t2_api" em Kettle

Validação: num teste com 5 pedidos, todas as respostas tiveram estado 200 e JSON com `numFound`. Na execução completa, 363 linhas lidas e 361 gravadas (2 duplicados removidos), com 105 "Sim", 255 "Não" e 1 "Desconhecido". A tabela `kettle.livros_final` e o Excel têm 361 livros (362 linhas com o cabeçalho). Face ao KNIME, os totais de "Não" (255) e de "Desconhecido" (1) coincidem, mas foram removidos 2 duplicados em vez de 3. Verificou-se que não restam obras repetidas com o mesmo título e autores; a diferença atribui-se, como hipótese, a alterações nos dados ou na ordem dos resultados da Open Library entre as duas execuções, uma vez que o identificador da obra vem da API.

5. Jobs

5.1 KNIME

No workflow principal, o processo está organizado em 9 metanós, agrupados visualmente pelas três fases do ETL (Extract, Transform e Load), e num nó de ligação à base de dados (PostgreSQL Connector), partilhado pelas etapas de carregamento.

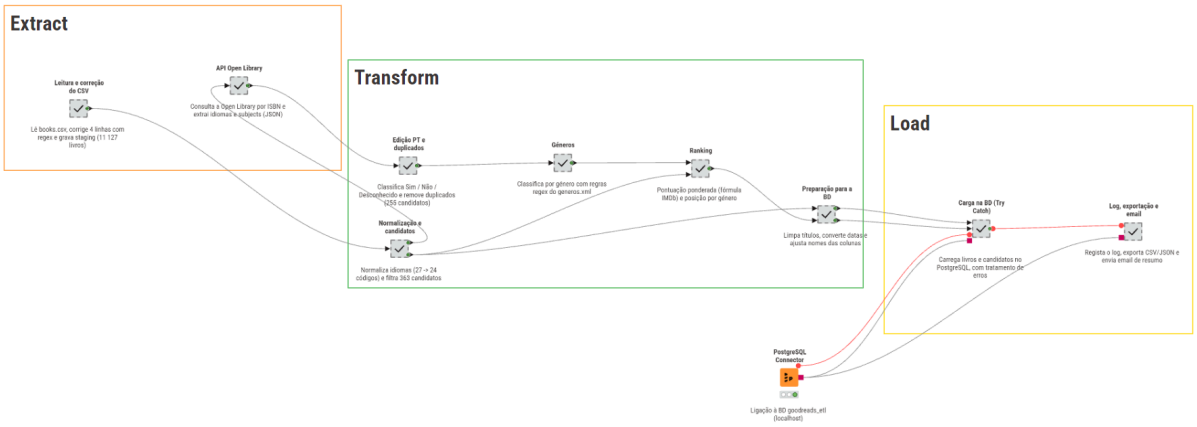


Figura 10: Workflow principal em KNIME, organizado por fases

A ordem de execução é garantida de duas formas:

- pelas **ligações de dados** entre os nós;
- por **ligações de flow variables**, quando não há dados a passar entre dois nós mas é preciso garantir a ordem. Por exemplo:
    - a leitura do ficheiro de staging só começa depois da sua escrita
    - os candidatos só são inseridos depois dos livros, por causa da chave estrangeira
    - o email só é enviado depois de o log e o ranking estarem escritos.

A carga na base de dados é completa (*full refresh*): antes de cada execução, as tabelas `livros` e `candidatos` são esvaziadas com `TRUNCATE`. Assim, o processo pode ser executado repetidamente sem duplicar dados. A tabela de log não é esvaziada e acumula uma linha por execução.

5.1.1 Carga na base de dados com tratamento de erros

A carga (o `TRUNCATE` e a inserção dos livros e dos candidatos) está protegida por um bloco Try / Catch Errors. Se algum nó dentro do bloco falhar, o Catch Errors apanha o erro e devolve a variável `FailingNode` com o nome do nó que falhou (“none” quando não há erro). Um Rule Engine Variable define, a partir dela, o estado da execução: “sucesso” ou “erro”. Os nós executados depois do bloco recebem a ligação diretamente do PostgreSQL Connector, para continuarem a funcionar mesmo quando a carga falha.

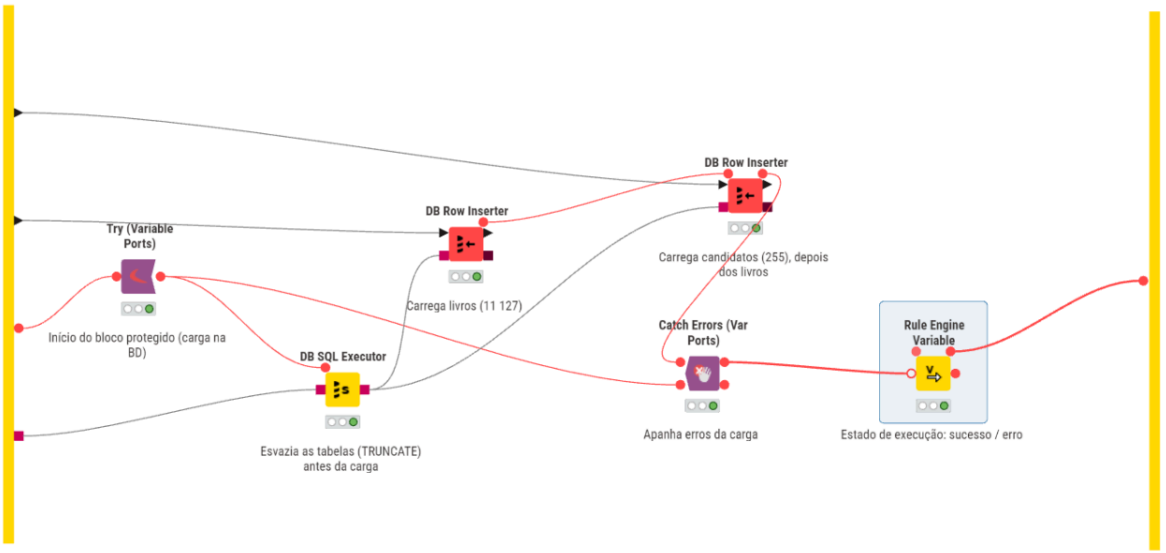


Figura 11: Metanó "Carga na BD (Try/Catch)"

Validação: foi forçado um erro, alterando temporariamente o nome da tabela de destino para uma tabela inexistente. O workflow não parou: registou no log o estado "erro" com a mensagem "Falhou no nó: DB Row Inserter" e enviou o email de alerta. Repondo o nome correto, a execução seguinte voltou a registar "sucesso".

5.1.2 Log, exportação e notificação

No fim de cada execução, o metanó final faz quatro coisas:

1. regista a execução na tabela `log_execucoes`, com as contagens obtidas diretamente da base de dados, o estado e uma mensagem
2. exporta o histórico de execuções para `data/output/logs/log_execucoes.csv`
3. lê o ranking da vista `v_ranking` e exporta-o para CSV e JSON
4. envia um email de resumo (SMTP Office 365, com STARTTLS), com o estado, as contagens e o ranking em anexo. Os valores da última execução são passados ao email como flow variables, e o nó Merge Variables garante que o email só é enviado quando o log e o ranking estão prontos. As credenciais do email ficam num nó Credentials Configuration, sem guardar a password no workflow.

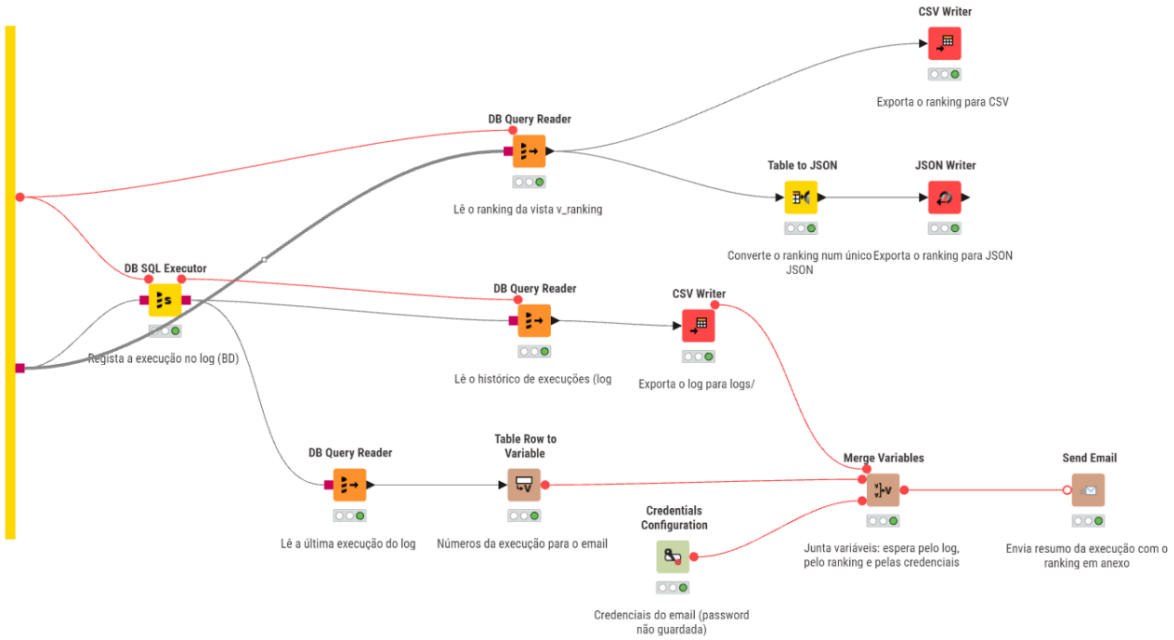


Figura 12: Metanó "Log, exportação e email"

Validação: o email foi recebido com os valores corretos (11 127 livros e 255 candidatos) e com o ranking em anexo; o log regista todas as execuções, incluindo a de erro forçado.

5.2 Dashboard em Node-RED

Os resultados são apresentados num dashboard desenvolvido em Node-RED, com o módulo Dashboard 2.0. O Node-RED lê diretamente da base de dados PostgreSQL e atualiza-se automaticamente a cada minuto. O painel tem três blocos:

- o número de candidatos por género (gráfico de barras)
- o melhor candidato de cada género
- o histórico das últimas execuções do processo, com o estado de cada uma.

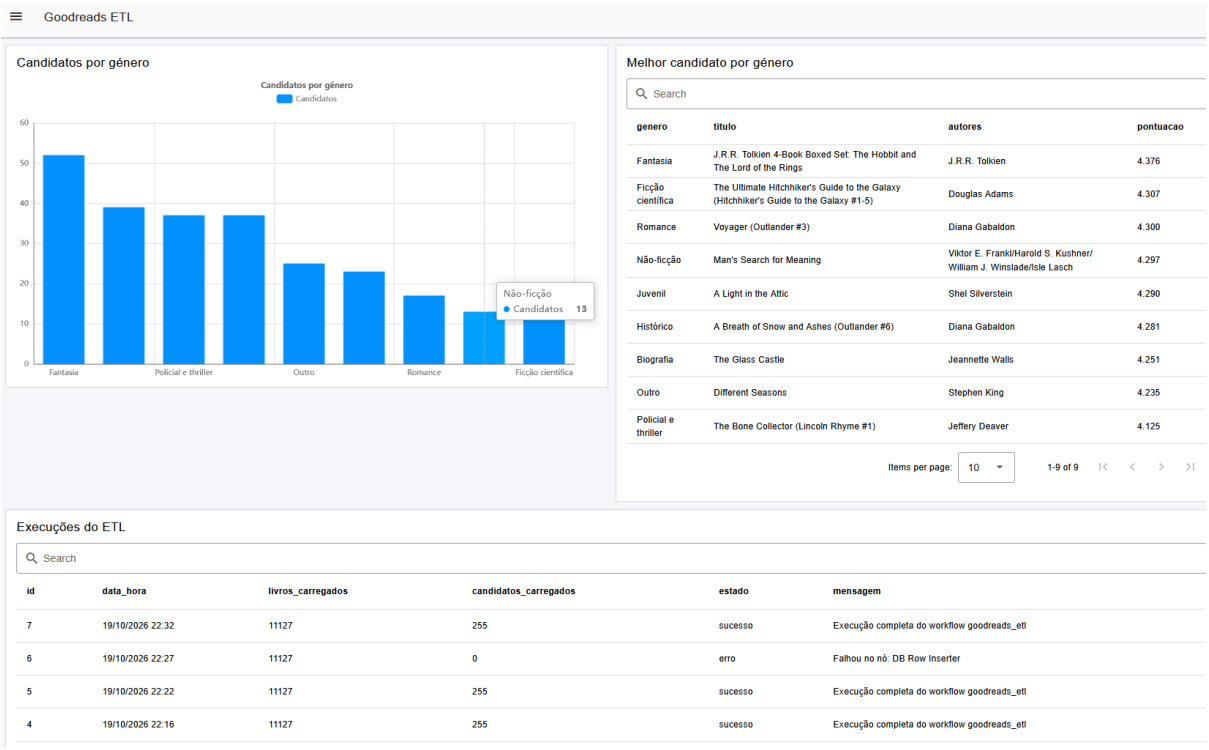


Figura 13: Dashboard "Goodreads ETL" em Node-RED

Durante o desenvolvimento foram resolvidos alguns problemas:

- a ligação sem SSL ao servidor local
- a conversão da contagem para inteiro, sem a qual o gráfico não era desenhado
- o modo "Replace" nas tabelas e no gráfico, para os dados não se repetirem a cada atualização.

5.3 Pentaho Data Integration (Kettle)

O job `job_etl.kjb` orquestra as duas transformações. Começa em `START`, executa `Executar t1` e `Executar t2` e termina com `Email sucesso` e `Success`. As ligações a verde ("Follow when result is true") garantem a ordem de execução: a `t2_api` só corre se a `t1_limpeza` terminar sem erros, o que é necessário porque lê a tabela `kettle.candidatos` que a `t1_limpeza` grava.

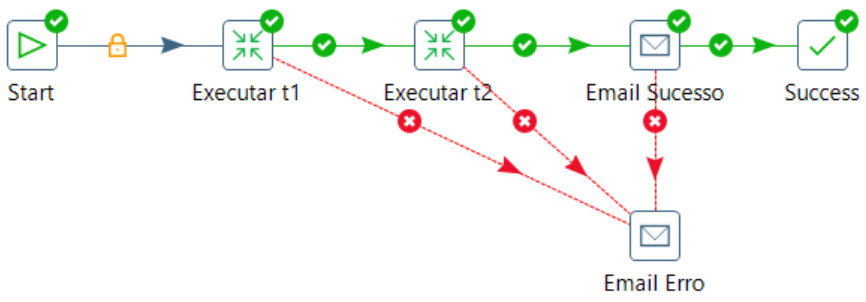


Figura 14: Job "job_etl" em Kettle

- **Email de sucesso:** o step `Email sucesso` envia uma mensagem por SMTP (Office 365, com TLS) quando as duas transformações terminam sem erros. O utilizador e a palavra-passe do email vêm de variáveis em `kettle.properties` (`SMTP_USER`, `SMTP_PASS`), e não ficam gravados no job.

- **Tratamento de erros:** de cada transformação (e do próprio email de sucesso) parte uma ligação a vermelho ("Follow when result is false") para o step `Email erro`, que envia um alerta com o log em anexo. Assim o job não falha em silêncio.

- **Logs:** cada entrada de transformação grava o seu log em ficheiro (`t1.log` e `t2.log`, em `data/output/logs`, nível Basic), e esse log segue em anexo nos emails.

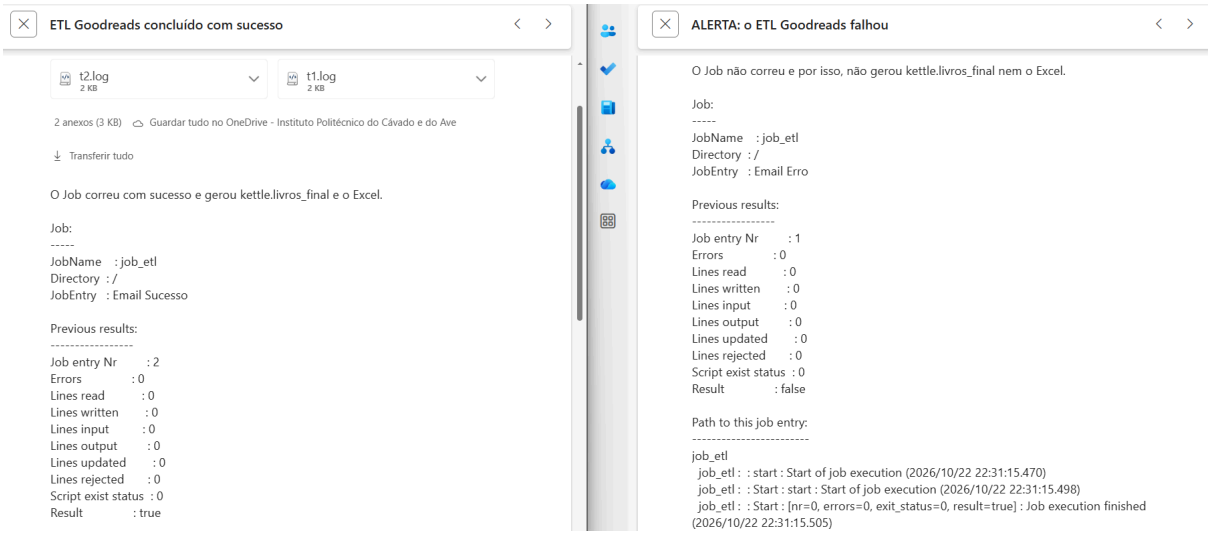


Figura 15: Emails de sucesso e de alerta enviados pelo job

Validação: a execução completa do job termina com todos os passos a verde, recebe-se o email de sucesso, e ficam 363 linhas em `kettle.candidatos`, 361 em `kettle.livros_final` e 362 linhas no Excel. O ramo de erro foi testado forçando uma falha (caminho do CSV inválido). O teste revelou um problema: o `Text file input` ignora por omissão os ficheiros em falta, pelo que a `t1_limpeza` lia 0 linhas, o job terminava com sucesso e o TRUNCATE esvaziava as tabelas. Marcando o ficheiro como obrigatório (*Required* = Y), a transformação passou a falhar, o fluxo seguiu pela ligação vermelha e o email de alerta foi enviado com o log em anexo.

6. Vídeo com demonstração

A demonstração do trabalho está dividida em dois vídeos, um por plataforma:

Parte 1: KNIME, PostgreSQL e Node-RED (Edgar Casal)

https://youtu.be/X_69v7VZAiQ

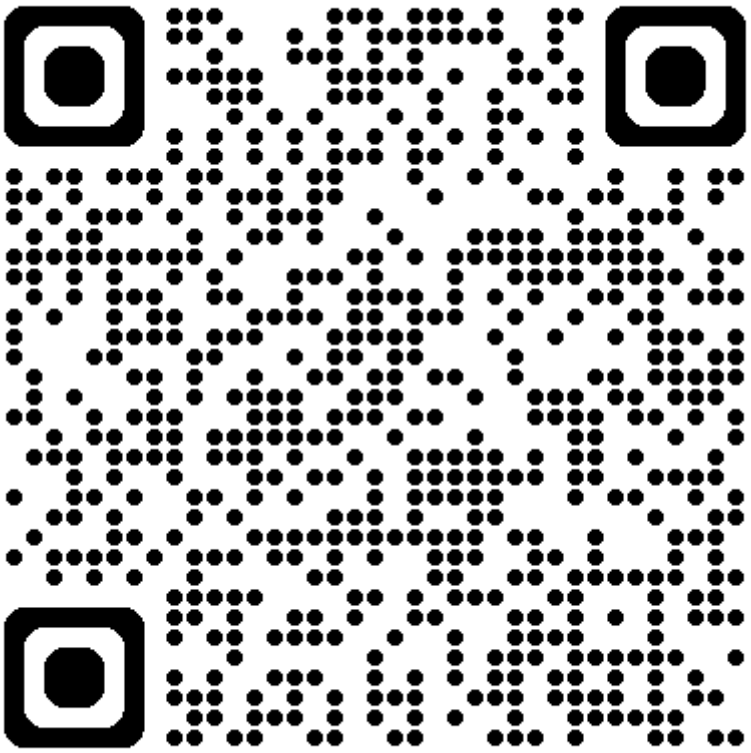


Figura 16: QR code do vídeo da parte desenvolvida em KNIME

- Este vídeo mostra o workflow em KNIME, os dados carregados no PostgreSQL, os ficheiros exportados, o email de resumo, o dashboard em Node-RED e o registo de uma execução com erro, tratada pelo bloco Try/Catch.

Parte 2: Pentaho Data Integration (Kettle) (João Simão Machado)

https://youtu.be/dqoOvzl8H5E



Figura 17: QR code do vídeo da parte desenvolvida em Kettle

- Este vídeo mostra as transformações e o job em Kettle, as tabelas no schema `kettle` e o ficheiro Excel exportado, o email de sucesso e o email de alerta de uma execução com erro.

7. Conclusão e trabalhos futuros

7.1 Conclusão

O trabalho permitiu desenvolver um processo ETL completo, que integra dados de fontes e formatos heterogéneos (um ficheiro CSV, uma API REST em JSON e um ficheiro de regras em XML) e os disponibiliza numa base de dados relacional, em ficheiros de exportação, por email e num dashboard.

A partir dos 11 127 livros do dataset do Goodreads, o processo:

- corrigiu os problemas de qualidade encontrados
- selecionou 363 livros populares e bem avaliados
- verificou na Open Library quais tinham edição em português
- removeu as edições duplicadas
- produziu uma pré-seleção de 255 candidatos a tradução, classificados em 9 géneros e ordenados por uma pontuação ponderada.

O processo pode ser executado repetidamente, regista cada execução e reage a erros, enviando um alerta por email.

Do ponto de vista da aprendizagem, destaca-se a importância de validar cada etapa com contagens que têm de bater certo. Foi assim que foram detetados problemas que de outra forma passariam despercebidos, como os falsos positivos nas expressões regulares. Destaca-se também o papel da base de dados como ponto de integração entre ferramentas diferentes (KNIME, Node-RED e Kettle).

Na comparação entre as duas ferramentas, ambas permitiram construir o processo e chegar a resultados muito próximos sobre os mesmos dados (os mesmos 363 candidatos e os mesmos totais de “Não” e “Desconhecido”). No Kettle, a separação entre transformações (fluxo de linhas) e job (orquestração) tornou a ordem de execução, o envio de emails de sucesso e de erro e o registo de logs em ficheiro muito diretos, através de ligações verdes e vermelhas, sem recorrer a flow variables nem a Try/Catch. Em contrapartida, exigiu mais configuração manual (driver JDBC do PostgreSQL, tipos de cada campo, um script JavaScript para juntar os idiomas das respostas JSON) e tem comportamentos silenciosos a que é preciso estar atento, como o `Text file input` aceitar um ficheiro em falta ou o TRUNCATE esvaziar a tabela mesmo quando a leitura falha. O KNIME, com os seus metanós e nós específicos (por exemplo, o JSON Path), foi mais prático para organizar e inspecionar um processo completo, enquanto o Kettle se mostrou mais leve para uma versão reduzida e mais orientado à execução repetida e automática.

7.2 Limitações

- **Cobertura da Open Library:** numa verificação manual de 7 livros classificados como “sem edição em português”, 6 tinham, de facto, edição em português (sobretudo brasileira). A classificação “Sim” é fiável, mas a classificação “Não” significa apenas que não existe registo na Open Library. Por isso, o resultado deve ser entendido como uma pré-seleção a verificar por um editor, e não como uma lista definitiva.
- **Coletâneas e edições especiais:** podem surgir como não traduzidas, mesmo quando as obras que as compõem estão traduzidas.
- **Classificação por género:** baseia-se em regras simples sobre os temas. Cerca de 10% dos livros ficaram sem género reconhecido (“Outro”), e, em caso de empate, é escolhido um dos géneros empatados.
- **Dados de origem:** o dataset é estático e não reflete livros nem avaliações recentes.

7.3 Trabalhos futuros

- Confirmar os livros classificados como “Não” com uma segunda fonte, por exemplo catálogos de bibliotecas ou de livrarias portuguesas.
- Pesquisar por título e autor quando a pesquisa por ISBN não devolve resultados.
- Afinar e alargar as regras de classificação por género.
- Acrescentar ao dashboard a escolha de um género, com o respetivo top 5.
- Agendar a execução automática do processo e passar de uma carga completa a uma carga incremental.
- Colocar os parâmetros do processo (limites de seleção, destinatários do email) num ficheiro de configuração externo.

8. Referências

- Ferreira, L., & Ribeiro, Ó. (2026). *Trabalho Prático I — Integração de Sistemas de Informação*. Escola Superior de Tecnologia, UPCA.
- Kaggle. (s.d.). *Goodreads-books* Dataset. https://www.kaggle.com/datasets/jealousleopard/goodreadsbooks
- Open Library. (s.d.). *Search API*. https://openlibrary.org/dev/docs/api/search
- KNIME AG. (s.d.). *KNIME Analytics Platform documentation*. https://docs.knime.com/
- The PostgreSQL Global Development Group. (s.d.). *PostgreSQL 18 documentation*. https://www.postgresql.org/docs/18/
- OpenJS Foundation. (s.d.). *Node-RED documentation*. https://nodered.org/docs/
- FlowFuse. (s.d.). *Node-RED Dashboard 2.0 documentation*. https://dashboard.flowfuse.com/
- Oracle. (s.d.). *Class Pattern (Java regular expressions)*. https://docs.oracle.com/javase/8/docs/api/java/util/regex/Pattern.html
- IMDb. (2017). *IMDb Vote FAQ*. Arquivado em https://web.archive.org/web/20171023205105/http://www.imdb.com/help/show_leaf?votestopfaq
- Hitachi Vantara. (s.d.). *Pentaho Data Integration documentation (versão 10.2)*. https://docs.pentaho.com/
- PostgreSQL Global Development Group. (s.d.). *PostgreSQL JDBC Driver*. https://jdbc.postgresql.org/